Introduction to Database Systems (CSD202)

Lab Assignment#5

Date: 15/02/19

You have to upload the screen shot (black screen) of output with the queries one by one, in a word doc & file name must be-> Roll No_Assignment#5.

1. Create and Display the tables employee & department (Drop both tables if already exist).

Structure forTable1: Employee

Attribute(Field/Column) Name	Data Declaration
Emp_ID	int-primary key
Emp_Name	Varchar(30) & it should be auto filled as **SNU Student** if we don't want to fill any value
Mgr_ID	Int

Structure for table 2: Department

Attribute(Field/Column) Name	Data Declaration
Dept_ID	Int, primary key
Location	Varchar(20) & it should be auto filed as **SNU** if we don't want to fill any value
Emp_ID	Int

2. Display both the tables after entering the data as given below (Do not insert Dept_ID in department table, it should be autofilled).

Note: - Fill data into both tables exactly the same as given below.

Table1: Employee

Emp_ID	Name	Mgr_ID
1001	Alex Bert	1003
1002		1003
1003	Mery Lee	1006
1004		1006
1005	Reyan Fert	1003
1006	Luther Warn	
1007	Amy	1003
1008	Coogan	1006

Table 2: Department

Dept_ID	Location	Emp_ID
200	Washington	1001
201	California	1002
202		1003
203	Houston	1004
204		1005
205	Washington	1006

Join Operations:

1. Display the matching record (Emp_ID, Name & Location) from both the tables.

2. Display the matching record (Emp_ID, Name & Location) records from both tables and all records from employee table.

3. Display the matching record (Emp_ID, Name & Location) records from both tables and all records from department table.

4. Display the list of employees who are only managers in ascending order (this is self-join).

5. Display the complete data (Emp_ID, Name & Location) from both the tables.

View:

1. Create a View **view1** having three columns Emp_id, Name & Location from both the tables.
2. Display View1.
3. Add one more column Dept_id to view1.
4. Display View1.
5. Drop view1.

Union, Union All, intersection & Difference:

1. Display the emp_id from both the tables (no duplicate record).
2. Display the emp_id from both the tables.
3. Perform an intersection operation & display the common employee details (Emp_ID, Name, Mgr_ID, Dept_ID and Location) from both the tables.
4. Perform an intersection operation & display the unmatched (not common) employee details (Emp_ID, Name, Mgr_ID) from both the tables.

   Note: - Although there is no INTERSECT,-,difference operator/keyword in MySQL, you can easily simulate this type of query using either the IN & NOT IN clause or the EXISTS & NOT EXISTS clause, depending on the complexity of the INTERSECT query. Here you can use EXISTS clause.

- See the difference between Union, union all, Intersection & difference after implementing above operations. You can implement these operations in many ways like w.r.t. to any common column from multiple tables & etc.)

Index:

1. Drop primary key from employee table & show the schema.
2. Create an index – First_Index on the table employee on filed Name (display the name of employees before creating the index and after creating the index).
3. Display the information of First_Index
4. Display the data of First_Index.
5. Drop the First_Index.

Note: - Drop table employee and department after completion.

ANSWERS

Join Operations:

Creating tables

```
mysql> create table employee
    -> (
    -> emp_id int,
    -> emp_name varchar(30) default 'SNU',
    -> mgr_id int,
    -> PRIMARY KEY (emp_id));
Query OK, 0 rows affected (0.18 sec)
```

```
mysql> describe employee;
+----------+-------------+------+-----+-------------+-------+
| Field    | Type        | Null | Key | Default     | Extra |
+----------+-------------+------+-----+-------------+-------+
| emp_id   | int(11)     | NO   | PRI | 0           |       |
| emp_name | varchar(30) | YES  |     | SNU Student |       |
| mgr_id   | int(11)     | YES  |     | NULL        |       |
+----------+-------------+------+-----+-------------+-------+
3 rows in set (0.02 sec)
```

```
mysql> create table dapartment
    -> (
    -> dept_id int auto_increment,
    -> location varchar(20) default 'SNU',
    -> emp_id int,
    -> primary key(dept_id));
Query OK, 0 rows affected (0.22 sec)
```

```
mysql> describe department;
+----------+-------------+------+-----+---------+----------------+
| Field    | Type        | Null | Key | Default | Extra          |
+----------+-------------+------+-----+---------+----------------+
| dept_id  | int(11)     | NO   | PRI | NULL    | auto_increment |
| location | varchar(20) | YES  |     | SNU     |                |
| emp_id   | int(11)     | YES  |     | NULL    |                |
+----------+-------------+------+-----+---------+----------------+
3 rows in set (0.03 sec)
```

Display Tables

```
mysql> select * from employee;
+--------+-------------+--------+
| emp_id | emp_name    | mgr_id |
+--------+-------------+--------+
|   1001 | Alex bert   |   1003 |
|   1002 | SNU Student |   1003 |
|   1003 | Mery Lee    |   1006 |
|   1004 | SNU Student |   1006 |
|   1005 | Reyan Fert  |   1003 |
|   1006 | Luther Warn |   NULL |
|   1007 | Amy         |   1003 |
|   1008 | Coogan      |   1006 |
+--------+-------------+--------+
8 rows in set (0.00 sec)
```

```
mysql> select * from department;
+---------+------------+--------+
| dept_id | location   | emp_id |
+---------+------------+--------+
|     200 | Washington |   1001 |
|     201 | California |   1002 |
|     202 | SNU        |   1003 |
|     203 | Houston    |   1004 |
|     204 | SNU        |   1005 |
|     205 | Washington |   1006 |
+---------+------------+--------+
6 rows in set (0.00 sec)
```

1. Display the matching record (Emp_ID, Name & Location) from both the tables.

```
mysql> select employee.emp_id,employee.emp_name,department.location
    -> from employee
    -> inner join department
    -> on employee.emp_id = department.emp_id;
+--------+------------+------------+
| emp_id | emp_name   | location   |
+--------+------------+------------+
|   1001 | Alex bert  | Washington |
|   1002 | SNU Student | California |
|   1003 | Mery Lee   | SNU        |
|   1004 | SNU Student | Houston    |
|   1005 | Reyan Fert | SNU        |
|   1006 | Luther Warn | Washington |
+--------+------------+------------+
6 rows in set (0.00 sec)
```

2. Display the matching record (Emp_ID, Name & Location) records from both tables and all records from employee table.

```
mysql> select employee.emp_id,employee.emp_name,department.location
    -> from employee
    -> left join department
    -> on employee.emp_id = department.emp_id;
+--------+-------------+------------+
| emp_id | emp_name    | location   |
+--------+-------------+------------+
|   1001 | Alex bert   | Washington |
|   1002 | SNU Student | California |
|   1003 | Mery Lee    | SNU        |
|   1004 | SNU Student | Houston    |
|   1005 | Reyan Fert  | SNU        |
|   1006 | Luther Warn | Washington |
|   1007 | Amy         | NULL       |
|   1008 | Coogan      | NULL       |
+--------+-------------+------------+
8 rows in set (0.00 sec)
```

3. Display the matching record (Emp_ID, Name & Location) records from both tables and all records from department table.

```
mysql> select employee.emp_id,employee.emp_name,department.location
    -> from employee
    -> right join department
    -> on employee.emp_id = department.emp_id;
+--------+-------------+------------+
| emp_id | emp_name    | location   |
+--------+-------------+------------+
|   1001 | Alex bert   | Washington |
|   1002 | SNU Student | California |
|   1003 | Mery Lee    | SNU        |
|   1004 | SNU Student | Houston    |
|   1005 | Reyan Fert  | SNU        |
|   1006 | Luther Warn | Washington |
+--------+-------------+------------+
6 rows in set (0.00 sec)
```

4. Display the list of employees who are only managers in ascending order (this is self-join).

```
mysql> select distinct a.emp_name as Name from employee a, employee b
    -> where a.emp_id = b.mgr_id;
+-------------+
| Name        |
+-------------+
| Mery Lee    |
| Luther Warn |
+-------------+
2 rows in set (0.00 sec)
```

5. Display the complete data (Emp_ID, Name & Location) from both the tables.

```
mysql> select employee.emp_id,employee.emp_name,department.location
    -> from employee
    -> cross join department;
+--------+-------------+------------+
| emp_id | emp_name    | location   |
+--------+-------------+------------+
|   1001 | Alex bert   | Washington |
|   1001 | Alex bert   | California |
|   1001 | Alex bert   | SNU        |
|   1001 | Alex bert   | Houston    |
|   1001 | Alex bert   | SNU        |
|   1001 | Alex bert   | Washington |
|   1002 | SNU Student | Washington |
|   1002 | SNU Student | California |
|   1002 | SNU Student | SNU        |
|   1002 | SNU Student | Houston    |
|   1002 | SNU Student | SNU        |
|   1002 | SNU Student | Washington |
|   1003 | Mery Lee    | Washington |
|   1003 | Mery Lee    | California |
|   1003 | Mery Lee    | SNU        |
|   1003 | Mery Lee    | Houston    |
|   1003 | Mery Lee    | SNU        |
|   1003 | Mery Lee    | Washington |
|   1004 | SNU Student | Washington |
|   1004 | SNU Student | California |
|   1004 | SNU Student | SNU        |
|   1004 | SNU Student | Houston    |
|   1004 | SNU Student | SNU        |
|   1004 | SNU Student | Washington |
|   1005 | Reyan Fert  | Washington |
|   1005 | Reyan Fert  | California |
|   1005 | Reyan Fert  | SNU        |
|   1005 | Reyan Fert  | Houston    |
|   1005 | Reyan Fert  | SNU        |
|   1005 | Reyan Fert  | Washington |
|   1006 | Luther Warn | Washington |
|   1006 | Luther Warn | California |
|   1006 | Luther Warn | SNU        |
|   1006 | Luther Warn | Houston    |
|   1006 | Luther Warn | SNU        |
|   1006 | Luther Warn | Washington |
|   1007 | Amy         | Washington |
|   1007 | Amy         | California |
|   1007 | Amy         | SNU        |
|   1007 | Amy         | Houston    |
|   1007 | Amy         | SNU        |
|   1007 | Amy         | Washington |
|   1008 | Coogan      | Washington |
|   1008 | Coogan      | California |
|   1008 | Coogan      | SNU        |
|   1008 | Coogan      | Houston    |
|   1008 | Coogan      | SNU        |
|   1008 | Coogan      | Washington |
+--------+-------------+------------+
48 rows in set (0.00 sec)
```

Union, Union All, intersection & Difference:

Display the emp_id from both the tables (no duplicate record).

```
mysql> select emp_id from employee
    -> UNION
    -> select emp_id from department;
+--------+
| emp_id |
+--------+
|   1001 |
|   1002 |
|   1003 |
|   1004 |
|   1005 |
|   1006 |
|   1007 |
|   1008 |
+--------+
8 rows in set (0.00 sec)
```

Display the emp_id from both the tables.

```
mysql> select emp_id from employee
    -> UNION ALL
    -> select emp_id from department;
+--------+
| emp_id |
+--------+
|   1001 |
|   1002 |
|   1003 |
|   1004 |
|   1005 |
|   1006 |
|   1007 |
|   1008 |
|   1001 |
|   1002 |
|   1003 |
|   1004 |
|   1005 |
|   1006 |
+--------+
14 rows in set (0.00 sec)
```

Perform an intersection operation & display the common employee details (Emp_ID, Name, Mgr_ID, Dept_ID and Location) from both the tables.

```
mysql> create view v1 as
    -> select emp_id from employee
    -> where emp_id in
    -> (select emp_id from department);
Query OK, 0 rows affected (0.03 sec)

mysql> select * from v1;
+--------+
| emp_id |
+--------+
|   1001 |
|   1002 |
|   1003 |
|   1004 |
|   1005 |
|   1006 |
+--------+
6 rows in set (0.00 sec)
```

```
mysql> select employee.emp_id,emp_name,mgr_id,dept_id,location from employee,department,v1
    -> where employee.emp_id=department.emp_id
    -> and employee.emp_id=v1.emp_id;
+--------+-------------+--------+---------+------------+
| emp_id | emp_name    | mgr_id | dept_id | location   |
+--------+-------------+--------+---------+------------+
|   1001 | Alex bert   |   1003 |     200 | Washington |
|   1002 | SNU Student |   1003 |     201 | California |
|   1003 | Mery Lee    |   1006 |     202 | SNU        |
|   1004 | SNU Student |   1006 |     203 | Houston    |
|   1005 | Reyan Fert  |   1003 |     204 | SNU        |
|   1006 | Luther Warn |   NULL |     205 | Washington |
+--------+-------------+--------+---------+------------+
6 rows in set (0.00 sec)
```

Perform an intersection operation & display the unmatched (not common) employee details (Emp_ID, Name, Mgr_ID, Dept_ID and Location) from both the tables.

```
mysql> create view v2 as
    -> select emp_id from employee
    -> where emp_id not in
    -> (select emp_id from department)
    -> UNION
    -> select emp_id from department
    -> where emp_id not in
    -> (select emp_id from employee);
Query OK, 0 rows affected (0.03 sec)

mysql> select * from v2;
+--------+
| emp_id |
+--------+
|   1007 |
|   1008 |
+--------+
2 rows in set (0.00 sec)
```

```
mysql> select employee.emp_id,emp_name,mgr_id from employee,v2
    -> where employee.emp_id=v2.emp_id;
+--------+----------+--------+
| emp_id | emp_name | mgr_id |
+--------+----------+--------+
|   1007 | Amy      |   1003 |
|   1008 | Coogan   |   1006 |
+--------+----------+--------+
2 rows in set (0.00 sec)
```

Index:

1. Drop primary key from employee table & show the schema.

```
mysql> alter table employee
    -> drop primary key;
Query OK, 8 rows affected (0.08 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> describe employee;
+----------+-------------+------+-----+-------------+-------+
| Field    | Type        | Null | Key | Default     | Extra |
+----------+-------------+------+-----+-------------+-------+
| emp_id   | int(11)     | NO   |     | NULL        |       |
| emp_name | varchar(30) | YES  |     | SNU Student |       |
| mgr_id   | int(11)     | YES  |     | NULL        |       |
+----------+-------------+------+-----+-------------+-------+
3 rows in set (0.00 sec)
```

2. Create an index – First_Index on the table employee on filed Name (display the name of employees before creating the index and after creating the index).

```
mysql> select emp_name as Names from employee;
+-------------+
| Names       |
+-------------+
| Alex Bert   |
| SNU Student |
| Mery Lee    |
| SNU Student |
| Reyan Fert  |
| Luther Warn |
| Amy         |
| Coogan      |
+-------------+
8 rows in set (0.00 sec)
```

```
mysql> create index first_index
    -> on employee
    -> (emp_name);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> select emp_name as Names from employee;
+-------------+
| Names       |
+-------------+
| Alex Bert   |
| Amy         |
| Coogan      |
| Luther Warn |
| Mery Lee    |
| Reyan Fert  |
| SNU Student |
| SNU Student |
+-------------+
8 rows in set (0.00 sec)
```

3. Display the information of First_Index

```
mysql> show index from employee;
+----------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table    | Non_unique | Key_name    | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| employee |          1 | first_index |            1 | emp_name    | A         |           7 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
+----------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
1 row in set (0.00 sec)
```

4. Display the data of First_Index.

I don't know what this means.

5. Drop the First_Index.

```
mysql> drop index first_index on employee;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> select emp_name from employee;
+-------------+
| emp_name    |
+-------------+
| Alex Bert   |
| SNU Student |
| Mery Lee    |
| SNU Student |
| Reyan Fert  |
| Luther Warn |
| Amy         |
| Coogan      |
+-------------+
8 rows in set (0.00 sec)
```